

Chapter 3

Software Estimation Technique

Contents:

Introduction:.....	2
Problems with over and underestimates:.....	7
Basis for successful estimating:	8
Software effort estimation Technique:	9
Expert Estimation:	9
Top-Down Estimation:.....	11
Bottom-Up Estimation:.....	12
Parametric Estimation:.....	13
Analogy:	16

Introduction:

Effort estimation is a crucial aspect of project management, playing a significant role in setting realistic timelines and allocating resources efficiently.

It involves predicting the amount of time and effort required to complete a particular task or project.

In software engineering, a big challenge is figuring out how long it will take to develop a software, how productive the team will be, and how good the quality of the final product will be.

The question is, can we use mathematical and scientific principles to make these predictions more objective and less based on personal opinions?

This question has sparked a debate. Some people, especially those who actually write code, often feel that making accurate and objective predictions about how long it will take to finish a project is almost impossible.

They argue that it's based on so many factors, and it's hard to predict exactly how things will go.

On the other side, there are many methods and processes in software development that claim they can provide objective estimates.

They **say they can predict** how complex a project will be, how long it will take, and how productive the programmers will be.

This debate is ongoing, and it's a bit like a **tug of war**. Some people believe that software development is inherently unpredictable, while others think that with the right methods and approaches, we can make more accurate and objective estimates.

What is effort estimation?

Effort estimation is the **process of forecasting** how much effort is required to develop or maintain a software application.

This effort is traditionally **measured in the hours worked by a person, or the money needed to pay** for this work.

Effort estimation is used to help draft project plans and budgets in the early stages of the software development life cycle.

This practice enables a project manager or product owner to accurately predict costs and allocate resources accordingly.

What makes a successful project?

Delivering:

- agreed functionality
- on time
- at the agreed cost
- with the required quality

Stages:

- set targets
- Attempt to achieve targets

Targets are set for a project and the project manager tries to meet them

A project manager has to produce :

- An estimate of the effort .
- An estimate of the activity durations

Some problems with estimating:

- Nature of software .
- Complexity and invisibility of software .
- Over-estimating small tasks and Under-estimating large ones .
- Political pressures
- Different objectives of people in an organization
- Managers may wish to reduce estimated costs in order to win support for acceptance of a project proposal
- Changing technologies
- Technology is rapidly changing , making the experience of previous project estimates difficult to use in new ones .
- Projects differ
- Experience on one project may not be applicable to another

Where are estimates done ?

Feasibility study

- Estimates here confirms that the benefits of the potential system will justify the costs

Project Planning:

- Estimates are often initiated during the project planning phase. Project managers and teams work together to estimate the time, resources, and costs required to complete the project successfully.

System Specification

- Design shows how user requirements will be fulfilled.
- Estimating The efforts needed to implement different design proposals.
- Estimates at the design stage will also confirm that the feasibility study is still valid.

Problems with over and underestimates:

An **over-estimate** is likely to cause project to take longer than it would otherwise

This can be explained by the application of two laws :

- Parkinson's Law: 'Work expands to fill the time available'

Thus , e.g . for an easy task over estimating the duration required to complete it will cause some staff to work less hard to fill the time .

- Brook's Law : putting more people on a late job makes it later

So overestimating the effort required to perform a task (activity) means more staff assigned to it than needed

Underestimating a project : Can cause the project to not be delivered on time or cost

but still could be delivered faster than a more generous estimate

On the other side the **danger of underestimating** a project is the **effect on the quality**

Zeroth law of reliability : if a system doesn't have to be reliable it can meet any other objective

Basis for successful estimating:

A. The need for historical data .

- Most estimating methods need information about past projects
- Care has to be considered when applying past performance to new projects because of possible differences in factors such as :
 - Different programming languages
 - Different experience of staff
 - Different terminology

B. Measuring work .

- The time and cost to implement software depends on :
 - The developer's capability and experience
 - The technology that will be used
- The usual practice is to start by expressing work size independently of the effort , using measures such as:
 - SLOC OR KLOC : Source lines of code or thousands of lines of code

- Alternative size measure is Function Points (FP)

Software effort estimation Technique:

1. Expert estimation
2. Top down estimation
3. Bottom-up estimation
4. Parametric estimation
5. Analogy

Expert Estimation:

- **Description:** Expert estimation means that an expert estimates how much effort a project requires.
 - Expert estimation relies on the **knowledge and experience of individuals or a group of experts** to provide estimates.
 - Experts use their judgment and familiarity with the domain to assess the effort required for a project or specific tasks.
 - The advantages of asking somebody else than the project manager to estimate a project, is that some

experts have deep knowledge about the problem at hand.

- The problem with expert estimates is that their reliability is typically unknown (unless the organisation tracks the performance of their estimators) and that their estimates are not objective.
- **Process:**
 - Experts may individually provide estimates based on their expertise.
 - Group discussions or consensus-building sessions may be held to refine estimates.
- **Pros:**
 - Valuable when historical data is limited.
 - Subjective insights from experienced professionals.
- **Cons:**
 - Susceptible to biases and individual variations.
 - May lack precision in the absence of data.

Top-Down Estimation:

- **Description:** Top-down estimation, also known as **analogous or high-level estimation**, involves using the **characteristics of a previous project as a reference to estimate the effort for a new project**. It's often done at the beginning of a project when detailed information is limited.
- **Process:**
 - Identify a similar past project.
 - Use historical data from that project to estimate the new project's effort.
- **Pros:**
 - Quick and requires less detailed information.
 - Useful when little is known about the current project.
- **Cons:**
 - Relies heavily on the similarity of projects.
 - May not account for unique project characteristics.

Bottom-Up Estimation:

- **Description:** Bottom-up estimation involves breaking down a project into its individual components, estimating each component separately, and then aggregating these estimates to get an overall project estimate. It is a detailed and granular approach.
- **Process:**
 - Break down the project into tasks or work packages.
 - Estimate each task independently.
 - Sum up individual estimates to obtain the total project estimate.
- **Pros:**
 - Provides a detailed understanding of project components.
 - Useful when the project is well-defined.
- **Cons:**
 - Time-consuming and requires a thorough understanding of tasks.
 - May overlook interactions between components.

Parametric Estimation:

- **Description:** Parametric estimation involves using statistical relationships between historical data and project parameters to calculate estimates. It is a quantitative method that relies on mathematical models.
- A formula for a parametric model :
$$\text{Effort} = (\text{System Size}) * (\text{Productivity Rate})$$
- In parametric estimation, **mathematical models and historical data are used** to establish a relationship between project characteristics (such as system size) and effort or cost.
- The **productivity rate represents the efficiency or productivity of the development process**, usually measured in terms of effort per unit of size.

Example: Estimating Effort for a Web Application Development Project

Suppose a software development team has historical data from past web application projects. They have observed a consistent relationship between the size of the web application (measured in thousands of lines of code) and the effort required to develop it. The team determines that,

on average, it takes 10 person-hours of effort for each thousand lines of code.

Now, they are tasked with estimating the effort required for a new web application project. The estimated size of the application is 50,000 lines of code.

Using the parametric estimation formula:

- $Effort = (SystemSize) \times (ProductivityRate)$
- $Effort = 50KLOC \times 10person-hours/KLOC$
- $Effort = 500person-hours$

In this example:

- System Size: 50,000 lines of code (KLOC)
- Productivity Rate: 10 person-hours per KLOC

The **estimated effort required** for the development of this web application is **500 person-hours**.

This estimation is based on the observed productivity rate derived from historical data.

- **Process:**

- Identify parameters (such as size, complexity, or productivity).
- Use historical data and regression analysis to create a formula for estimation.

- **Pros:**

- Provides a more objective and formulaic approach.
- Can be scalable to various project sizes.

- **Cons:**

- Assumes a stable relationship between parameters and effort.
- Requires historical data and statistical expertise.

Analogy:

- **Description:** It is also called **case-based reasoning** .
 - For a new project the **estimator identifies the previous completed projects** that have similar characteristics to it .
 - The **new project** is referred to as the **target project or target case**
 - The **completed projects** are referred to as the **source projects** or source case
 - The effort recorded for the **matching source case** is used as the **base estimate** for the target project
 - The estimator calculates an estimate for the new project by adjusting the (base estimate) **based on the differences that exist between the two projects**
 - There are software tools that automate this process by selecting the nearest project cases to the new project .
 - Some software tools perform that by measuring are Euclidean distance between cases (projects).
 - The Euclidean distance is calculated as follows :
 - distance = **square-root of ((target_parameter1 - source_parameter1)²+ (target_parameter n - source_parameter n)²)**

Question:

Assume that cases are matched on the basis of two parameters , the number of inputs and the number of outputs .

- The new project (target case) requires 7 inputs and 15 output
- You are looking into two past cases (source cases) to find a better analogy with the target project :
- Project A : has 8 inputs and 17 outputs .
- Project B : has 5 inputs and 10 outputs .

Which is a more closer match for the new project A or project B ?

Distance between new project and project A :

$$\text{Square-root of } ((7-8)^2 + (15-17)^2) = 2.24$$

Distance between new project and project B :

$$\text{Square-root of } ((7-5)^2 + (15-10)^2) = 5.39$$

Project A is a better match because it has less distance than project B to the new project

• Process:

- Identify projects with similarities to the current project.
- Use the historical data from those projects to estimate the current project.

- **Pros:**

- Benefits from past project experiences.
- Allows for adjustments based on observed differences.

- **Cons:**

- Highly dependent on the availability of relevant historical data.
- Subject to biases and assumptions.